

Name: Nirupom Saha
Reg. No: 2141019011

Computer Architecture - Software
Assignment - 2

Scenario: A Hypothetical E-Commerce Platform

To build a scalable and maintainable online shopping platform, we used Microservices Architecture to divide the system into separate services and Event Driven Architecture (EDA) for real-time communication and responsiveness.

Microservices Architecture in E-commerce platform:

E-commerce uses a microservices architecture, with each functional unit divided into autonomous services.

- User Service: Handles registration, login and user profiles.
- Product Catalogue Service: Manages product listings and inventories.
- Order Service: Handles customer orders and payments.
- Shipping Service: Handles delivery and tracking.
- Notification Service: Sends order status and promotional messages.

Each service may scale independently and be launched individually, allowing for continuous supply.

Event - Driven Architecture :

Events enable asynchronous communication between services via a message broker (e.g., Kafka).

- When an order is placed, Order Service generates the OrderPlaced event.
- Inventory Service listens for OrderPlaced to minimise inventory.
- The Notification Service listens and provides confirmation to the consumer.
- Shipping Service takes up OrderConfirmed to begin delivery.

This decouples the services, boosts performance, and builds resiliency.

SOLID Principles :

- S - Single Responsibility Principle : Each service focusses on a certain domain (e.g., Product Service alone handles catalogue management).
- O - Open/Closed Principles : This principle allows for the addition of new features, such as wishlists, coupons or discounts, without affecting existing services.
- L - Liskov Substitution Principle : The Liskov Substitution principle states that interfaces for message consumers/publishers should be testable with fake versions.
- I - Interface Segregation Principle : Services only offer interfaces appropriate to specific clients. For example, the Admin UI utilises larger APIs than the Customer UI.

• D- Dependency Inversion Principle : This principle states that high-level services, such as Checkout, be based on event contracts rather than direct implementations of other services.

DRY Principles (Don't Repeat Yourself) :

- Common libraries or utility services include shared functionality for logging, authentication and configuration.
- Event payload schemas are specified centrally and shared across services.

• KISS Principles (Keep it Simple, Stupid) :

- Services have clear and focused logic, avoiding unnecessary stacking or nesting.
- Each service just exposes relevant endpoints and consumes necessary events.
- Allows services to respond to events spontaneously, reducing the need for needless orchestration.

Using Microservices and Event-Driven Architecture in E-Commerce platform improves modularity, and responsiveness. By implementing SOLID, DRY, and KISS, the system stays clean, manageable, and expandable without becoming unduly complicated.